

Auth.101

Sheryians · Cohort

FOUNDATIONS

1 The problem

2 Authn vs authz

3 Validation

TOKENS

4 The ID card

5 Inside a JWT

6 The payload

7 The full flow

PASSWORDS

8 Why we hash

9 bcrypt in practice

IMPLEMENTATION

10 Setup

11 Register route

12 Login route

13 Auth middleware

14 Storing the token

WRAP UP

15 Common mistakes

COHORT · BACKEND · SESSION NOTES

Who sent **this** request?

Your server already accepts requests and talks to a database. But every request that arrives looks the same — the server has no idea which human is on the other end. Today we fix that, and then decide what each identified user is allowed to do.

Validation

Authentication

Authorization

JWT

bcrypt

Express + Mongoose

01 The problem — who sent this request?

Before we write a single line of auth code, feel the problem it solves

Three users are using your app right now — A, B and C. All three tap the same button, so all three send the same request to the same route on your server.

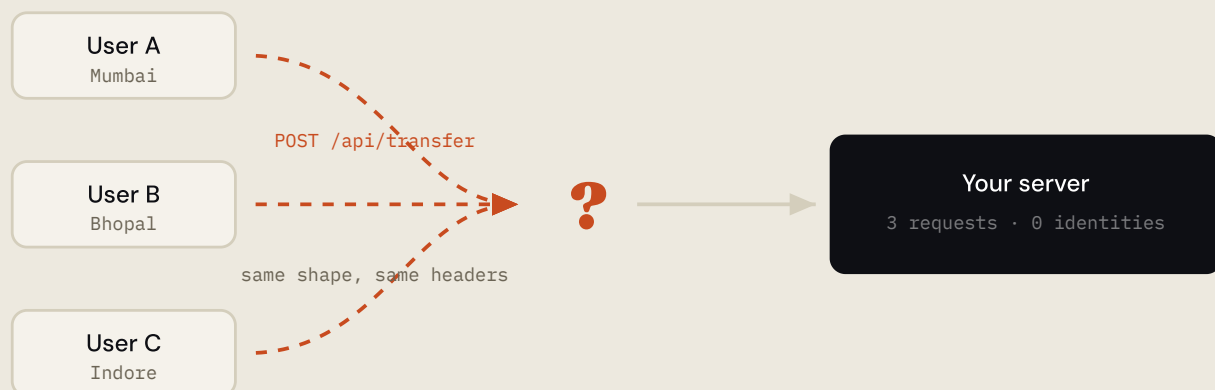


FIG 1 — THE SERVER RECEIVES REQUESTS, NOT PEOPLE

The server sees three HTTP requests. It does not see A, B or C. HTTP is **stateless** — the server keeps no memory of who connected last time, and nothing in a plain request says who is holding the phone.

KEY CONCEPT

Stateless means every request is treated as a brand new stranger. Request number two knows nothing about request number one. If we want the server to recognise a user, the *request itself* has to carry the proof.

Why this actually matters

Take a banking app. A user asks to transfer ₹60,000:

INCOMING REQUEST

COPY

POST /api/transfer

Content-Type: application/json

```
{
  "to": "XYZ0001234",
  "amount": 60000
}
```

Transfer 60,000 to XYZ0001234 — fine.

But FROM whose account? The request never says.

Money has to leave someone's account. Without identity this endpoint is either useless or catastrophic. The same question shows up everywhere: whose cart is this, whose posts do I return, whose profile is being edited.



The obvious fix — **send `userId` in the body** — is the classic beginner trap. The body is written by the client. Anyone can open Postman and put *your* id in it and empty your account. **Never trust data the client can edit.** Identity must be something the client cannot fake.

DEFINITION

Authentication is the process of identifying the user who made the request.

Why can't the server simply remember who user A is after their first request?

A — Because the database is too slow to store it

B — Because HTTP is stateless — each request arrives with no memory of the last one

02 Authentication vs authorization

Two different questions that students mix up in every interview

These two words look alike and are constantly confused. They answer completely different questions, and they run one after the other.

KEY CONCEPT

Authentication asks *who are you?* · **Authorization** asks *are you allowed to do this?* You cannot check what someone is allowed to do until you know who they are — so authentication always runs first.

What	Authentication	Authorization
The question	Who is this user?	Is this user allowed to do it?
Runs	First	After authentication
Checks	Credentials or a valid token	Role, ownership, permissions
Fails with	401 Unauthorized — "I don't know you"	403 Forbidden — "I know you, and no"
In our app	Verifying the token on <code>/api/orders</code>	Only <code>role: "admin"</code> may delete a product

A real example from a college: the guard at the gate checks your ID card and confirms you are a student — that is authentication. Inside, that same card does not open the staff room, because the card says *student*, not *teacher* — that is authorization.



401 vs 403 is a favourite interview question. 401 means no valid identity was presented. 403 means the identity is fine, the permission is not.

A logged-in normal user hits `DELETE /api/products/12`, an admin-only route. What should the server respond?

A — 401 Unauthorized, because the request must be blocked

B — 403 Forbidden, because we know who they are but their role isn't allowed

03 Validation — check before you trust

The step that runs before we even ask who you are

Before identity, sanity. **Validation** asks a simpler question: is this data even usable? Is the email actually shaped like an email, is the password long enough, is `amount` a positive number and not the string `"sixty thousand"`?

ORDER OF A PROTECTED REQUEST

● Validation

Is the incoming data the right shape? → 400 Bad Request

● Authentication

Who is sending this? → 401 Unauthorized

● Authorization

Is this user allowed here? → 403 Forbidden

● Controller

Only now do we touch the database and do the real work.

```
// the simplest possible validation – do it before anything else
const { name, email, password } = req.body;

if (!name || !email || !password) {
  return res.status(400).json({ message: "name, email and password are required" });
}

if (password.length < 8) {
  return res.status(400).json({ message: "password must be at least 8 characters" });
}

// In a real project you would not write this by hand for every route.
// Use a schema library – zod, joi or express-validator – as middleware.
```



The validation you wrote in React is **UX, not security**. Postman, curl and a script never open your form. If the only check lives in the browser, there is effectively no check. **Validate on the server, always.**

You already block empty emails in your React form. Do you still need the same check in Express?

A — Yes — anyone can bypass the UI and call the API directly

B — No — it is already handled, checking twice is wasted work

04 The student ID card

The whole token idea, in a story you have already lived

Think about joining a school or college.

First you fill an **admission form** — name, details, documents. The office verifies it and writes you into their register. A few days later they hand you an **ID card**. That card carries your roll number, your class, a photo, an expiry date, and the school's stamp or hologram.

Now watch what the card does. At the library gate, the computer lab, the swimming pool, the sports club, the ground — you show the card and you are let in. Nobody sends you back to the admission office. Nobody makes you fill the form again. The card *is* your proof, and every gate can check it on the spot.

That is exactly what a token does for your server.

At school	In our backend
Filling the admission form	Registration — <code>POST /api/auth/register</code>
Office verifies and records you	Server validates and saves the user in the database
School issues an ID card	Server signs and issues a JWT
Roll number printed on the card	The user's <code>id</code> inside the token payload
Stamp / hologram nobody can copy	The signature, created with a secret only the server has
Showing the card at the library gate	Sending the token with every request
Card valid till March 2027	<code>expiresIn</code> — the token stops working after that
Lost card? Report it, get a new one	Logout / re-login issues a fresh token

KEY CONCEPT

The gatekeeper never calls the admission office. Everything needed to trust you is **printed on the card itself**, and the hologram proves it is genuine. That is why JWT auth is called **stateless** — the server does not have to store a session for you anywhere.



Push the analogy one more step and you get authorization for free: a **student card** opens the library, a **teacher card** opens the staff room. Same card system, different role printed on it.

In the ID card analogy, what plays the role of the school's hologram?

A — The payload, because it holds the user's details

B — The signature, because only the server's secret can produce it

05 Inside a JWT

Three parts, two dots, and one very common misunderstanding

JWT stands for JSON Web Token. It is a long string that looks like random noise, but it is not random at all. Look for the two dots — they split it into three parts.

A REAL JWT

COPY

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4ZjNhMWM5NGU3YjJkMDAxMmM5YWI0NSIsInJvbm90IjoiZm9udC91b3R5IiwiaWF0IjoiMTYxMjM0MDAwIn0=.eyJpZCI6IjY4ZjNhMWM5NGU3YjJkMDAxMmM5YWI0NSIsInJvbm90IjoiZm9udC91b3R5IiwiaWF0IjoiMTYxMjM0MDAwIn0=.eyJpZCI6IjY4ZjNhMWM5NGU3YjJkMDAxMmM5YWI0NSIsInJvbm90IjoiZm9udC91b3R5IiwiaWF0IjoiMTYxMjM0MDAwIn0=
```

header . payload . signature

PART	HOLDS	WHAT IT IS FOR
Header	{ alg, typ }	Which algorithm signed this token. Usually HS256, and the type is JWT.
Payload	{ id, role, iat, exp }	The actual data — the "claims". This is what your server reads to know who the user is.
Signature	HMACSHA256(...)	Header + payload signed with your secret key. This is the hologram — proof the token came from your server and was not edited.

Decode it yourself

The first two parts are just JSON, Base64URL-encoded. No key, no password, no special tool — the browser can read them. Paste any token below and see it open up.

JWT DECODER — PASTE A TOKEN

DECODE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4ZjNhMWM5NGU3YjJkMDAxMmM5YWI0NSIsInJvbmUiOiJzdHVkZW50IiwiaWF0IjoxNzU2NzAzNDAwLCJleHAiOjE3NTczMDgyMDB9.Jz74pc6s1u5dKsQ0WBkuKcfGXT4Mz3IVg2GiJBvDaYU
```

DECODE TOKEN

LOAD STUDENT TOKEN

LOAD EDITED TOKEN

CLEAR

HEADER

DECODED

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD

READABLE BY ANYONE

```
{
  "id": "68f3a1c94e7b2d0012c9ab45",
  "role": "student",
  "iat": 1756703400,
  "exp": 1757308200
}
```

iat → issued at 1 Sept 2025, 10:40 am

exp → expires 8 Sept 2025, 10:40 am (already expired)

SIGNATURE

NEEDS THE SECRET

Jz74pc6s1u5dKsQ0WBkuKcfGXT4Mz3IVg2GiJBvDaYU

The server recomputes this from the header and payload using JWT_SECRET.
Change one character above and this no longer matches – `jwt.verify()` throws.



Base64URL is **encoding, not encryption**. A JWT is readable by anyone who has it — the user, a browser extension, anyone reading a log file. **Never put a password, OTP, card number or Aadhaar inside a token.**

Then what stops someone editing it?

Try it live: press **Load edited token** above. Someone took the student token, changed `"role":` `"student"` to `"role": "admin"`, and re-encoded the payload. It decodes perfectly. It looks like a valid admin token.

But the signature was calculated from the *original* header and payload using a secret that only your server knows. Change one character of the payload and the signature no longer matches. When your server runs `jwt.verify()`, it recomputes the signature and sees the mismatch — the token is rejected.

Attacker edits payload → role: "admin"

↓

Server recomputes signature using its secret

↓

Computed signature ≠ signature on the token

↓

`jwt.verify()` throws → 401 Unauthorized ✓

KEY CONCEPT

A JWT is **readable by everyone, forgeable by no one**. Read access is public. Write access needs the secret. That single line explains almost every JWT rule you will ever be told.

Your teammate says "the payload is safe, it's encrypted". Are they right?

A — Yes — you need the secret key to read a JWT payload

B — No — it is only Base64URL encoded, anyone holding the token can read it

An attacker edits the payload of a token they stole. What happens on the server?

A — Signature verification fails, because the signature was made from the original payload

06 What goes inside the payload

Keep it small, keep it boring

Since the payload is public, the rule writes itself: put in the minimum the server needs to identify the user, and nothing else. In practice that is the user's **id**, and — if you have roles — the **role**.

Field	Put it in the token?	Why
<code>id</code> / <code>_id</code>	Yes	The whole point — it tells the server which database document this request belongs to.
<code>role</code>	Yes	Lets authorization run without an extra DB call. Not secret information.
<code>iat</code> , <code>exp</code>	Automatic	Issued-at and expiry. The library adds these for you when you pass <code>expiresIn</code> .
Password or its hash	Never	It would be public. This is the single worst JWT mistake.
Phone, address, Aadhaar, DOB	No	Personal data that anyone holding the token could read. Fetch it from the DB when needed.
Cart, orders, big objects	No	The token travels with every single request. Keep it light.

UTILS/TOKEN.JS

COPY

```
const jwt = require("jsonwebtoken");

function signToken(user) {
  return jwt.sign(
    { id: user._id, role: user.role }, // payload - small on purpose
    process.env.JWT_SECRET,           // secret - never hardcode it
  );
}
```

```
    { expiresIn: "7d" } // adds iat + exp for you
  );
}

module.exports = { signToken };
```



A token is a **photocopy taken at login time**, not a live view of the database. If you promote a user to admin today, their old token still says **student** until it expires — and if you demote an admin, their old token still says **admin**. That is exactly why tokens should be short-lived.

A student wants to store the user's phone number in the token to avoid a database call. What do you tell them?

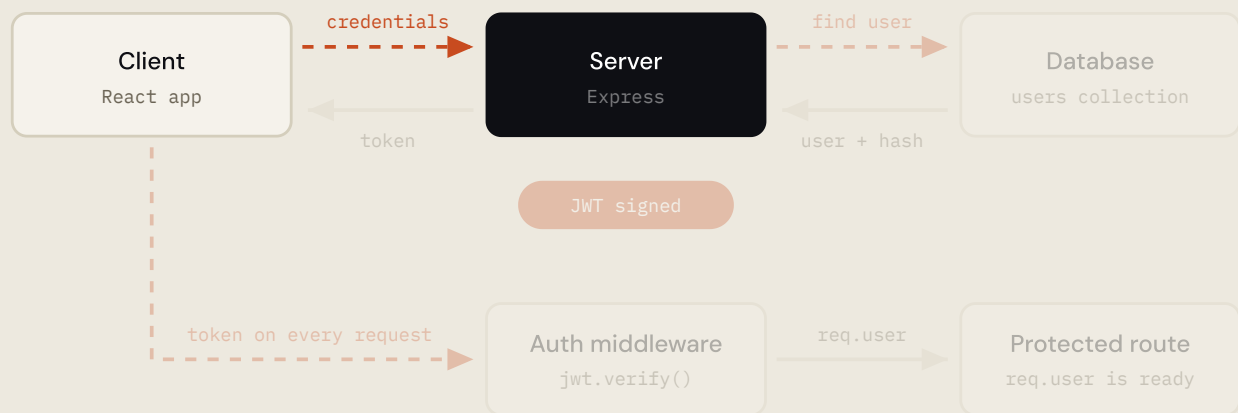
A — Don't — the payload is public, so personal data must not go in it

B — Fine — the signature protects everything inside the payload

07 The complete flow, step by step

Walk it forward and backward until the whole class can narrate it

Everything so far in one journey. Use the buttons to move through it — one step at a time.



1 • Login request

The client sends email and password to POST /api/auth/login. This is the only moment the real password travels over the network.

[< BACK](#)

Step **1** / 7

[FORWARD >](#)

Steps 1 to 5 happen **once**, at login. Steps 6 and 7 happen on **every single request** after that. That difference is the whole reason tokens exist — the password is checked once, the card is shown a thousand times.

08 Why we hash passwords

The one mistake that can end a product and a career

Registration means storing a user in your database. The tempting version looks like this:

USERS COLLECTION — THE DANGEROUS VERSION

COPY

```
{ "email": "riya@gmail.com", "password": "riya@1234" }
{ "email": "aman@gmail.com", "password": "sheryians123" }
{ "email": "neha@gmail.com", "password": "sheryians123" }
```

It works. Login is a simple ==. And it is a disaster waiting to happen.

Now list everyone who can read that column

- Anyone who gets a copy of the database — a leak, a stolen backup, a public Mongo instance with no password on it.
- Every developer, intern and DBA on your team, including the one who leaves next month.
- Whatever log, analytics tool or error report accidentally captured a request body.

And here is the part students underestimate. People reuse passwords. `sheryians123` is probably also Neha's email password, her Instagram password, maybe her bank password. If your little project leaks, you did not just lose your app's accounts — you handed over other people's lives. That is why this is treated as a serious offence and not a bug.

KEY CONCEPT

Hashing is a one-way street. You can turn a password into a hash, but you cannot turn a hash back into the password. So we never store the password — we store the hash. At login we hash the incoming password again and compare the two hashes. The server itself never knows your real password.

What	Encoding (Base64)	Encryption (AES)	Hashing (bcrypt)
Reversible?	Yes, by anyone	Yes, with the key	No, by design
Needs a key?	No	Yes	No
Used for	Safe transport of data — JWT parts	Data you must read back later	Passwords — you never need them back

Why bcrypt and not MD5 or SHA-256

MD5 and SHA-256 are hashes too, but they were built to be *fast*. A modern GPU can try billions of guesses per second against them, and precomputed rainbow tables already contain every common password. Fast is exactly what you do not want here.

bcrypt is deliberately **slow**, and you control how slow with a cost factor. It also **salts** every password automatically — a random value mixed in before hashing. Look at Aman and Neha, who chose the same password:

```
BCRYPT.HASH("SHERYIANS123", 10) — RUN TWICE
```

COPY

```
# aman
$2b$10$GnnIMosW7ua0VRg98b9j0eBlf6ZQb6lmq1cLpTZ09y6fD/SgMypMe

# neha - same password, completely different hash
$2b$10$Ao8zemGb5v7Wbp/o8E.V/eGHZrT0K.m.doIcku8ZxQpFNCUojLMqUu
```

Same input, different output, because each got its own random salt. An attacker cannot spot that two users share a password, and cannot crack a thousand accounts with one precomputed table — every account has to be attacked separately, slowly.



Never hash on the client, never log a password, and never send the password or its hash back in a response. And do not invent your own hashing scheme — **reversing your own "secret formula" is the first thing an attacker tries.**

Two users pick the same password. Why do their bcrypt hashes look completely different?

A — Because bcrypt encrypts each one with a different secret key

B — Because bcrypt mixes a fresh random salt into every hash

09 bcrypt in practice

Two functions, and what that weird string actually contains

STEP 1 – HASH ON REGISTRATION

HASHING A PASSWORD

```
// store hashedPassword in the database – never the plain one
```

STEP 2 – COMPARE ON LOGIN

CHECKING A PASSWORD

[COPY](#)

```
const isMatch = await bcrypt.compare(password, user.password);  
//                                     plain input   stored hash  
  
if (!isMatch) {  
  return res.status(401).json({ message: "invalid email or password" });  
}
```

Notice we never "decrypt" anything. `bcrypt.compare` reads the salt out of the stored hash, hashes the incoming password with that same salt, and checks whether the results match.

READING A BCRIPT HASH

SEGMENT	VALUE IN OUR EXAMPLE	WHAT IT MEANS
algorithm	\$2b	Which bcrypt version produced this hash.
cost	\$10	2 ¹⁰ rounds of work. Every +1 doubles the time to hash — and to attack.
salt	GnnIMosW7ua0VRg98b9j0e	The 22-character random salt, stored right inside the hash so login can reuse it.
hash	B1f6ZQb61mq1cLpTZo9y6fD/SgMypMe	The actual result of hashing password + salt.



Cost **10** is a sensible default — roughly 50–100 ms per hash. Slow enough to make brute force painful, fast enough that your login route still feels instant. Do not push it to 20 "for extra security" unless you enjoy 30-second logins.

Where does bcrypt store the salt it used?

A — Inside the hash string itself, so `compare()` can read it back

B — In a separate salts collection you have to create

10 Setup

Packages, environment variables, and the user model

TERMINAL

COPY

```
npm install jsonwebtoken bcryptjs cookie-parser dotenv
```

PACKAGE	USED FOR	WHAT IT GIVES YOU
<code>jsonwebtoken</code>	<code>tokens</code>	Creates and verifies JWTs — <code>jwt.sign()</code> and <code>jwt.verify()</code> .
<code>bcryptjs</code>	<code>passwords</code>	Hashes passwords and compares them — <code>bcrypt.hash()</code> and <code>bcrypt.compare()</code> .
<code>cookie-parser</code>	<code>reading cookies</code>	Fills <code>req.cookies</code> so the middleware can pick the token out of the request.
<code>dotenv</code>	<code>secrets</code>	Loads your <code>.env</code> file into <code>process.env</code> .

.ENV

COPY

```
PORT=3000
MONGO_URI=mongodb://127.0.0.1:27017/auth-class
JWT_SECRET=a-long-random-string-nobody-can-guess
```



Your `JWT_SECRET` is the hologram die. Anyone who has it can mint a token for any user, including an admin. **Keep it in `.env`, add `.env` to `.gitignore`, and never hardcode it or push it to GitHub.**

MODELS/USER.MODEL.JS

COPY

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
```

```

name:      { type: String, required: true, trim: true },
email:     { type: String, required: true, unique: true, lowercase: true },
password:  { type: String, required: true, select: false },
role:      { type: String, enum: ["user", "admin"], default: "user" }
}, { timestamps: true });

module.exports = mongoose.model("User", userSchema);

```



`select: false` on the password means every normal query leaves it out automatically. You cannot leak a hash you never fetched. On login you ask for it explicitly with `.select("+password")`.

11 The register route

Filling the admission form

CONTROLLERS/AUTH.CONTROLLER.JS

COPY

```

const bcrypt = require("bcryptjs");
const User = require("../models/user.model");
const { signToken } = require("../utils/token");

module.exports.register = async (req, res) => {
  const { name, email, password } = req.body;

  // 1. validate
  if (!name || !email || !password)
    return res.status(400).json({ message: "all fields are required" });

  // 2. is this email already admitted?
  const exists = await User.findOne({ email });
  if (exists)
    return res.status(409).json({ message: "email already registered" });

  // 3. hash - the plain password must never reach the database
  const hashedPassword = await bcrypt.hash(password, 10);

  // 4. create the user (the register entry)
  const user = await User.create({ name, email, password: hashedPassword });

```

```
// 5. issue the ID card
const token = signToken(user);
res.cookie("token", token, { httpOnly: true });

return res.status(201).json({
  message: "registered successfully",
  user: { id: user._id, name: user.name, email: user.email }
}); // ↑ never send the password back
};
```



Notice the order: **validate** → **check duplicate** → **hash** → **save** → **issue token**. Every step is cheap protection for the step after it. And **409 Conflict** is the honest status for "this email already exists", not **400**.

12 The login route

Collecting a fresh ID card

CONTROLLERS/AUTH.CONTROLLER.JS

COPY

```
module.exports.login = async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password)
    return res.status(400).json({ message: "email and password are required" });

  // password is select:false in the schema, so ask for it explicitly
  const user = await User.findOne({ email }).select("+password");

  if (!user)
    return res.status(401).json({ message: "invalid email or password" });

  const isMatch = await bcrypt.compare(password, user.password);

  if (!isMatch)
    return res.status(401).json({ message: "invalid email or password" });

  const token = signToken(user);
  res.cookie("token", token, { httpOnly: true });
};
```

```
return res.status(200).json({ message: "logged in" });  
};
```



Both failures return the same message: **"invalid email or password"**. If you say "user not found" for one and "wrong password" for the other, you have just built a tool that tells an attacker which emails are registered on your platform.

13 The auth middleware

The guard at every gate

This is where authentication actually pays off. One small middleware sits in front of every protected route, checks the card, and hands the route a ready-made `req.user`.

MIDDLEWARES/AUTH.MIDDLEWARE.JS

COPY

```
const jwt = require("jsonwebtoken");  
  
module.exports.isLoggedIn = (req, res, next) => {  
  // the card can arrive as a cookie or an Authorization header  
  const token =  
    req.cookies.token || req.headers.authorization?.split(" ")[1];  
  
  if (!token)  
    return res.status(401).json({ message: "please login first" });  
  
  try {  
    // verify = recompute the signature and check expiry  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    req.user = decoded;      // { id, role, iat, exp }  
    next();  
  } catch (err) {  
    return res.status(401).json({ message: "invalid or expired token" });  
  }  
};
```



Use `jwt.verify()`, never `jwt.decode()`. `decode()` only reads the payload and checks nothing — no signature, no expiry. Using it to authenticate means any string a user invents becomes a valid login.

Adding authorization on top

Authentication is done. Now the role check — the same card, a different gate:

MIDDLEWARES/ROLE.MIDDLEWARE.JS

COPY

```
module.exports.allow = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ message: "you are not allowed to do this" });
  next();
};
```

ROUTES/PRODUCT.ROUTES.JS

COPY

```
// authentication → authorization → controller
router.delete(
  "/products/:id",
  isLoggedIn,           // who are you? → 401
  allow("admin"),       // allowed to do it? → 403
  deleteProduct
);

// inside deleteProduct you can now trust req.user.id — it came from
// a signed token, not from the request body.
```

Why must the controller use `req.user.id` instead of `req.body.userId`?

A — `req.user.id` came from a verified token; the body is written by the client

B — They are the same, `req.user.id` is just shorter to type

14 Where the token lives

Two places to keep the card, and why one is safer

What	localStorage	httpOnly cookie
Sent automatically	No — you attach it by hand on every call	Yes, the browser attaches it
Readable by JavaScript	Yes — so any injected script can steal it	No — JS cannot touch it
Works with	Mobile apps, third-party APIs	Browser apps on your own domain
Needs care about	XSS	CSRF — handled with <code>sameSite</code>

PRODUCTION COOKIE OPTIONS

COPY

```
res.cookie("token", token, {
  httpOnly: true,           // JS cannot read it
  secure: process.env.NODE_ENV === "production", // HTTPS only
  sameSite: "lax",         // blocks most CSRF
  maxAge: 7 * 24 * 60 * 60 * 1000 // 7 days, matching the JWT
});

// logout = take the card back
res.clearCookie("token");
```



Here is the honest limitation of stateless auth: because the server stores nothing, it cannot un-issue a token. If someone steals a valid token, it works until it expires — clearing the cookie only clears *that browser*. The industry answer is short-lived access tokens plus refresh tokens, which is our next session.

Why is an `httpOnly` cookie safer than `localStorage` for a browser app?

A — Because cookies are encrypted by the browser

15 Mistakes to avoid

Check your project against this list before you push

Mistake	What it costs you	Do this instead
Storing plain passwords	One leak exposes every user, and their other accounts too	Always <code>bcrypt.hash(password, 10)</code>
Personal data in the payload	Anyone holding the token can read it	Only <code>id</code> and <code>role</code>
Hardcoding <code>JWT_SECRET</code>	Secret ends up on GitHub; anyone can mint admin tokens	<code>.env</code> plus <code>.gitignore</code>
Using <code>jwt.decode()</code> to authenticate	No signature check — auth is completely bypassed	<code>jwt.verify()</code> inside try/catch
Trusting <code>req.body.userId</code>	Any user can act as any other user	<code>req.user.id</code> from the middleware
Tokens that never expire	A stolen token is valid forever	Set <code>expiresIn</code> , keep it short
"User not found" vs "wrong password"	Leaks which emails are registered	One message for both
Returning the user with the hash	Ships the hash to the browser for no reason	<code>select: false</code> , send only safe fields

16 Recap and practice

What to build before the next class

Request arrives – server has no idea who sent it



Validate the body – is this data even usable?



Authenticate – verify the token, set `req.user` (who are you?)



Authorize – check the role (are you allowed?)



Controller runs with a trusted identity ✓

Build this yourself

1 – Register and login

A fresh Express + Mongoose project with `/register` and `/login`. Hash with bcrypt, issue a JWT in an httpOnly cookie. Check in Compass that no plain password was ever saved.

2 – Protect a route

Add `GET /profile` behind `isLoggedIn`, returning the logged-in user from `req.user.id`. Call it in Postman without a token and confirm you get 401.

3 – Add a role

Give one user `role: "admin"`, add `DELETE /products/:id` behind `allow("admin")`, and confirm a normal user gets 403 while the admin gets through.

4 – Break it on purpose

Copy your token into the decoder in section 05, change the role to admin, re-encode it and send it. Watch the server reject it – then explain out loud why.

QUESTIONS YOU SHOULD BE ABLE TO ANSWER

- What is the difference between authentication and authorization, and which status code belongs to each?
- Why is a JWT not secure to put personal data in, even though it is signed?
- Why do we hash passwords instead of encrypting them?
- What does a salt do, and why do two identical passwords produce different hashes?
- What is the difference between `jwt.decode()` and `jwt.verify()`?
- Why can't a stateless JWT be cancelled before it expires?